Name : Nishagiri Kamble

Roll No : CO2032

~Sonar Rock vs Mine Detection using Machine Learning

Problem Statement

The aim of this project is to build a Machine Learning model that can predict whether an object detected by sonar is a Rock or a Mine based on sonar signal data. This helps in underwater object detection and improves safety in naval operations.

Dataset Information

- Dataset Name: Sonar Dataset

- Number of Records: 208

- Number of Features: 60

- Target Column: Label (R = Rock, M = Mine)

- Dataset Type: Classification

In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

In [4]:
```
import os

os.getcwd()
```

Out[4]:
```
'D:\\python Ai IOT\\All Datasets'
```

In [7]:
```
import os
os.listdir()
```

Out[7]:
```
['.ipynb_checkpoints',
 '2 ML_Notes (1).docx',
 '4Data_Cleaning_Missing_Values_Outliers.pdf',
 '5Matplotlib_Tutorial.pdf',
 '6 Feature_Encoding_Complete.pdf',
 '8 feature_engineering using titanic dataset.pdf',
 '9 titanic_feature_train_testsplit.pdf',
 'advertising.ipynb',
 'All Datasets',
 'B3 Parkinson Dataset.zip',
 'B6 ECG Arthamania.zip',
 'B9 PCB Fault detection.zip',
 'EDAAllDatasets.ipynb',
 'Sonar.ipynb',
 'StrokePrediction.ipynb']
```

In [8]:
```
import os

os.listdir("All Datasets")
```

Out[8]:
```
['.ipynb_checkpoints',
 'Advertising.csv',
 'Bahaman.xlsx',
 'Bolt diameter.xlsx',
 'Boston.csv',
 'claimants sample.csv',
 'claimants.csv',
 'Contract Renewal.xlsx',
 'Fabric data.xlsx',
 'gene_expression.csv',
 'hearing_test.csv',
 'homeprices.csv',
 'Indian Liver Patient Dataset -Description.txt',
 'JohnyTalkers.xlsx',
 'liver.csv',
 'liver.xlsx',
 'mouse_viral_study.csv',
 'mushrooms.csv',
 'penguins_size.csv',
 'Promotion.xlsx',
 'readDatasets.ipynb',
 'sonar.csv',
 'sonar.names.txt',
 'stroke prediction.csv',
 'tips.csv',
 'titanic.csv',
 'train_test.csv',
 'Untitled.ipynb',
 'Untitled1.ipynb']
```

Data Loading

In [89]:
```
df = pd.read_csv("All Datasets/sonar.csv", header=None)
```

In [90]:
```
df
```

Out[90]:

	0	1	2	3	4	5	6	7	8	9	...	
0	0.0200	0.0371	0.0428	0.0207	0.0954	0.0986	0.1539	0.1601	0.3109	0.2111	...	0.00
1	0.0453	0.0523	0.0843	0.0689	0.1183	0.2583	0.2156	0.3481	0.3337	0.2872	...	0.00
2	0.0262	0.0582	0.1099	0.1083	0.0974	0.2280	0.2431	0.3771	0.5598	0.6194	...	0.02
3	0.0100	0.0171	0.0623	0.0205	0.0205	0.0368	0.1098	0.1276	0.0598	0.1264	...	0.01
4	0.0762	0.0666	0.0481	0.0394	0.0590	0.0649	0.1209	0.2467	0.3564	0.4459	...	0.00
...	...	...	...	...	...	...	...	...	...	...	...	
203	0.0187	0.0346	0.0168	0.0177	0.0393	0.1630	0.2028	0.1694	0.2328	0.2684	...	0.01
204	0.0323	0.0101	0.0298	0.0564	0.0760	0.0958	0.0990	0.1018	0.1030	0.2154	...	0.00
205	0.0522	0.0437	0.0180	0.0292	0.0351	0.1171	0.1257	0.1178	0.1258	0.2529	...	0.01
206	0.0303	0.0353	0.0490	0.0608	0.0167	0.1354	0.1465	0.1123	0.1945	0.2354	...	0.00
207	0.0260	0.0363	0.0136	0.0272	0.0214	0.0338	0.0655	0.1400	0.1843	0.2354	...	0.01

208 rows × 61 columns

In [91]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 208 entries, 0 to 207
Data columns (total 61 columns):
 #   Column  Non-Null Count  Dtype  
---  ------  --------------  -----  
 0   0       208 non-null    float64
 1   1       208 non-null    float64
 2   2       208 non-null    float64
 3   3       208 non-null    float64
 4   4       208 non-null    float64
 5   5       208 non-null    float64
 6   6       208 non-null    float64
 7   7       208 non-null    float64
 8   8       208 non-null    float64
 9   9       208 non-null    float64
 10  10      208 non-null    float64
 11  11      208 non-null    float64
 12  12      208 non-null    float64
 13  13      208 non-null    float64
 14  14      208 non-null    float64
 15  15      208 non-null    float64
 16  16      208 non-null    float64
 17  17      208 non-null    float64
 18  18      208 non-null    float64
 19  19      208 non-null    float64
 20  20      208 non-null    float64
 21  21      208 non-null    float64
 22  22      208 non-null    float64
 23  23      208 non-null    float64
 24  24      208 non-null    float64
 25  25      208 non-null    float64
 26  26      208 non-null    float64
 27  27      208 non-null    float64
 28  28      208 non-null    float64
 29  29      208 non-null    float64
 30  30      208 non-null    float64
 31  31      208 non-null    float64
 32  32      208 non-null    float64
 33  33      208 non-null    float64
 34  34      208 non-null    float64
 35  35      208 non-null    float64
 36  36      208 non-null    float64
 37  37      208 non-null    float64
 38  38      208 non-null    float64
 39  39      208 non-null    float64
 40  40      208 non-null    float64
 41  41      208 non-null    float64
 42  42      208 non-null    float64
 43  43      208 non-null    float64
 44  44      208 non-null    float64
 45  45      208 non-null    float64
 46  46      208 non-null    float64
 47  47      208 non-null    float64
 48  48      208 non-null    float64
 49  49      208 non-null    float64
 50  50      208 non-null    float64
```

```
 51  51      208 non-null    float64
 52  52      208 non-null    float64
 53  53      208 non-null    float64
 54  54      208 non-null    float64
 55  55      208 non-null    float64
 56  56      208 non-null    float64
 57  57      208 non-null    float64
 58  58      208 non-null    float64
 59  59      208 non-null    float64
 60  60      208 non-null    object
dtypes: float64(60), object(1)
memory usage: 99.3+ KB
```

In [92]:
```
df.shape
```

Out[92]:
```
(208, 61)
```

In [93]:
```
df.head()
```

Out[93]:

	0	1	2	3	4	5	6	7	8	9	...	51
0	0.0200	0.0371	0.0428	0.0207	0.0954	0.0986	0.1539	0.1601	0.3109	0.2111	...	0.0027
1	0.0453	0.0523	0.0843	0.0689	0.1183	0.2583	0.2156	0.3481	0.3337	0.2872	...	0.0084
2	0.0262	0.0582	0.1099	0.1083	0.0974	0.2280	0.2431	0.3771	0.5598	0.6194	...	0.0232
3	0.0100	0.0171	0.0623	0.0205	0.0205	0.0368	0.1098	0.1276	0.0598	0.1264	...	0.0121
4	0.0762	0.0666	0.0481	0.0394	0.0590	0.0649	0.1209	0.2467	0.3564	0.4459	...	0.0031

5 rows × 61 columns

Data Cleaning

In [94]:
```
df.isnull().sum()
```

Out[94]:
```
0     0
1     0
2     0
3     0
4     0
     ..
56    0
57    0
58    0
59    0
60    0
Length: 61, dtype: int64
```

Data Preproccesing

In [95]:
```
df.describe().sum()
```

Out[95]:

0	208.262455
1	208.401096
2	208.500860
3	208.665545
4	208.739280
5	208.849476
6	208.901585
7	209.046576
8	209.469390
9	209.626950
10	209.787518
11	209.833668
12	209.927092
13	210.327793
14	210.427978
15	210.662862
16	210.888235
17	211.040773
18	211.277625
19	211.593749
20	211.752478
21	211.805858
22	211.951950
23	212.047071
24	212.064975
25	212.221669
26	212.190512
27	212.126563
28	211.893349
29	211.616702
30	211.210218
31	210.914577
32	210.876633
33	210.785125
34	210.759478
35	210.688994
36	210.573819
37	210.518456
38	210.440049
39	210.320118
40	210.205988
41	210.065472
42	209.861110
43	209.699891
44	209.526286
45	209.414044
46	209.081931
47	208.731066
48	208.425558
49	208.171314
50	208.171627
51	208.130154
52	208.086794
53	208.083617
54	208.085428
55	208.075583

```
56    208.069481
57    208.078469
58    208.071023
59    208.072963
dtype: float64
```

In [96]:
```
df.duplicated().sum()
```

Out[96]:
```
0
```

In [97]:
```
df.dtypes
```

Out[97]:
```
0     float64
1     float64
2     float64
3     float64
4     float64
       ...
56    float64
57    float64
58    float64
59    float64
60     object
Length: 61, dtype: object
```

In [98]:
```
df.describe()
```

Out[98]:

	0	1	2	3	4	5	6
count	208.000000	208.000000	208.000000	208.000000	208.000000	208.000000	208.000000
mean	0.029164	0.038437	0.043832	0.053892	0.075202	0.104570	0.121747
std	0.022991	0.032960	0.038428	0.046528	0.055552	0.059105	0.061788
min	0.001500	0.000600	0.001500	0.005800	0.006700	0.010200	0.003300
25%	0.013350	0.016450	0.018950	0.024375	0.038050	0.067025	0.080900
50%	0.022800	0.030800	0.034300	0.044050	0.062500	0.092150	0.106950
75%	0.035550	0.047950	0.057950	0.064500	0.100275	0.134125	0.154000
max	0.137100	0.233900	0.305900	0.426400	0.401000	0.382300	0.372900

8 rows × 60 columns

In [99]:
```
df.columns
```

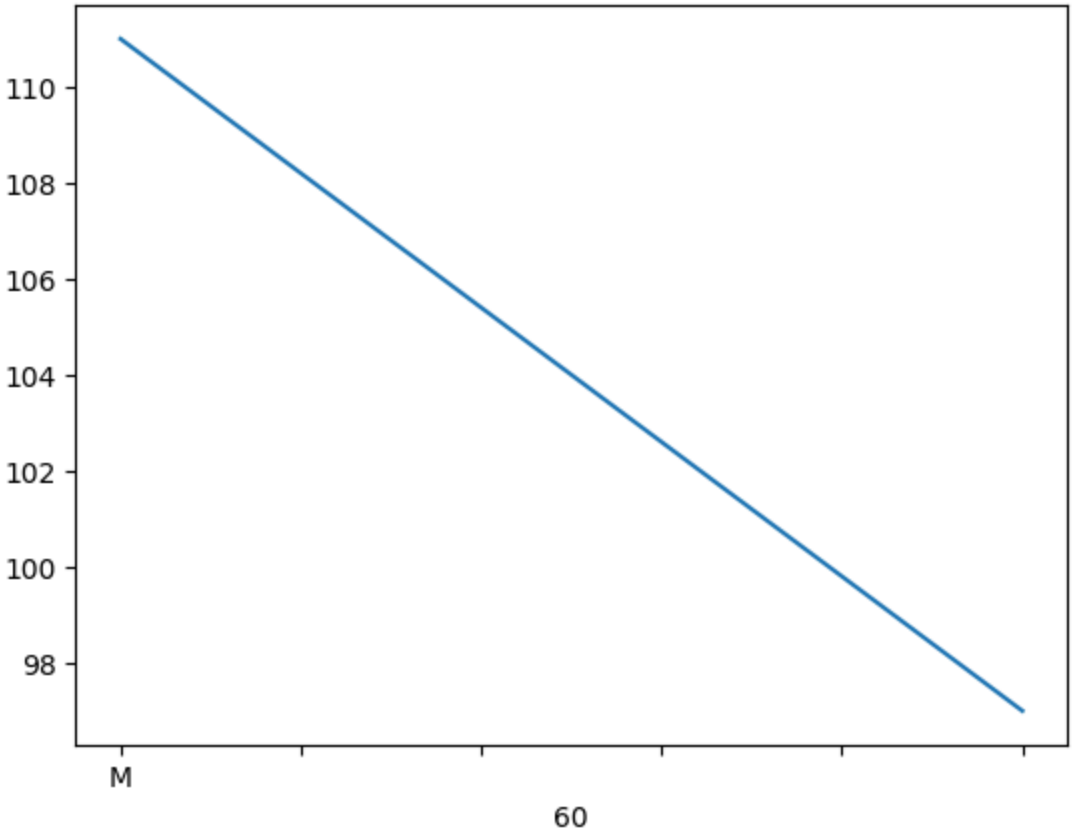
Out[99]:
```
Index([ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11, 12, 13, 14, 15, 16, 17,
       18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35,
       36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 46, 47, 48, 49, 50, 51, 52, 53,
       54, 55, 56, 57, 58, 59, 60],
      dtype='int64')
```

Exploratory Data Analysis

Bar plot

In [100…

```
df[60].value_counts().plot()
plt.show()
```

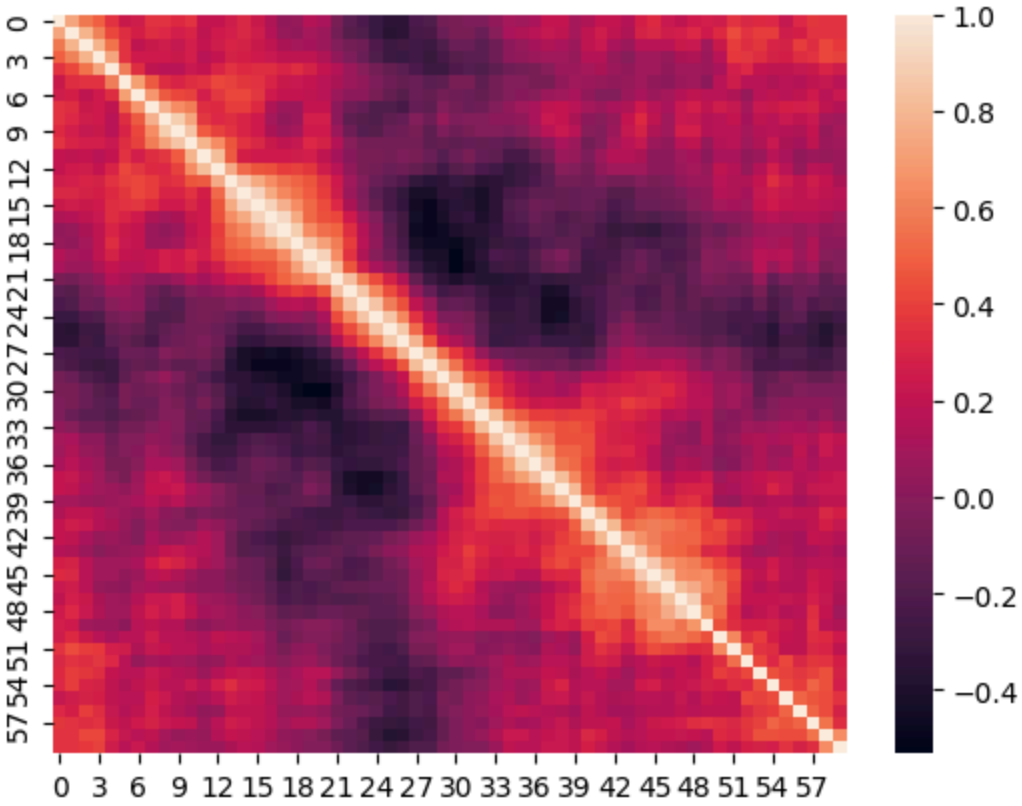


Correction Heatmap

In [101…

```
sns.heatmap(df.corr(numeric_only=True))


plt.show()
```



In [102…

```
df.tail
```

Out[102…

```
<bound method NDFrame.tail of          0       1       2       3       4       5
6       7       8   \
0    0.0200  0.0371  0.0428  0.0207  0.0954  0.0986  0.1539  0.1601  0.3109
1    0.0453  0.0523  0.0843  0.0689  0.1183  0.2583  0.2156  0.3481  0.3337
2    0.0262  0.0582  0.1099  0.1083  0.0974  0.2280  0.2431  0.3771  0.5598
3    0.0100  0.0171  0.0623  0.0205  0.0205  0.0368  0.1098  0.1276  0.0598
4    0.0762  0.0666  0.0481  0.0394  0.0590  0.0649  0.1209  0.2467  0.3564
..      ...     ...     ...     ...     ...     ...     ...     ...     ...
203  0.0187  0.0346  0.0168  0.0177  0.0393  0.1630  0.2028  0.1694  0.2328
204  0.0323  0.0101  0.0298  0.0564  0.0760  0.0958  0.0990  0.1018  0.1030
205  0.0522  0.0437  0.0180  0.0292  0.0351  0.1171  0.1257  0.1178  0.1258
206  0.0303  0.0353  0.0490  0.0608  0.0167  0.1354  0.1465  0.1123  0.1945
207  0.0260  0.0363  0.0136  0.0272  0.0214  0.0338  0.0655  0.1400  0.1843

         9   ...      51      52      53      54      55      56      57  \
0    0.2111  ...  0.0027  0.0065  0.0159  0.0072  0.0167  0.0180  0.0084
1    0.2872  ...  0.0084  0.0089  0.0048  0.0094  0.0191  0.0140  0.0049
2    0.6194  ...  0.0232  0.0166  0.0095  0.0180  0.0244  0.0316  0.0164
3    0.1264  ...  0.0121  0.0036  0.0150  0.0085  0.0073  0.0050  0.0044
4    0.4459  ...  0.0031  0.0054  0.0105  0.0110  0.0015  0.0072  0.0048
..      ...  ...     ...     ...     ...     ...     ...     ...     ...
203  0.2684  ...  0.0116  0.0098  0.0199  0.0033  0.0101  0.0065  0.0115
204  0.2154  ...  0.0061  0.0093  0.0135  0.0063  0.0063  0.0034  0.0032
205  0.2529  ...  0.0160  0.0029  0.0051  0.0062  0.0089  0.0140  0.0138
206  0.2354  ...  0.0086  0.0046  0.0126  0.0036  0.0035  0.0034  0.0079
207  0.2354  ...  0.0146  0.0129  0.0047  0.0039  0.0061  0.0040  0.0036

         58      59  60
0    0.0090  0.0032   R
1    0.0052  0.0044   R
2    0.0095  0.0078   R
3    0.0040  0.0117   R
4    0.0107  0.0094   R
..      ...     ...  ..
203  0.0193  0.0157   M
204  0.0062  0.0067   M
205  0.0077  0.0031   M
206  0.0036  0.0048   M
207  0.0061  0.0115   M

[208 rows x 61 columns]>
```

In [103…

```
df.head
```

Out[103…

```
<bound method NDFrame.head of          0       1       2       3       4       5
6       7       8   \
0    0.0200  0.0371  0.0428  0.0207  0.0954  0.0986  0.1539  0.1601  0.3109
1    0.0453  0.0523  0.0843  0.0689  0.1183  0.2583  0.2156  0.3481  0.3337
2    0.0262  0.0582  0.1099  0.1083  0.0974  0.2280  0.2431  0.3771  0.5598
3    0.0100  0.0171  0.0623  0.0205  0.0205  0.0368  0.1098  0.1276  0.0598
4    0.0762  0.0666  0.0481  0.0394  0.0590  0.0649  0.1209  0.2467  0.3564
..      ...     ...     ...     ...     ...     ...     ...     ...     ...
203  0.0187  0.0346  0.0168  0.0177  0.0393  0.1630  0.2028  0.1694  0.2328
204  0.0323  0.0101  0.0298  0.0564  0.0760  0.0958  0.0990  0.1018  0.1030
205  0.0522  0.0437  0.0180  0.0292  0.0351  0.1171  0.1257  0.1178  0.1258
206  0.0303  0.0353  0.0490  0.0608  0.0167  0.1354  0.1465  0.1123  0.1945
207  0.0260  0.0363  0.0136  0.0272  0.0214  0.0338  0.0655  0.1400  0.1843

         9   ...      51      52      53      54      55      56      57  \
0    0.2111  ...  0.0027  0.0065  0.0159  0.0072  0.0167  0.0180  0.0084
1    0.2872  ...  0.0084  0.0089  0.0048  0.0094  0.0191  0.0140  0.0049
2    0.6194  ...  0.0232  0.0166  0.0095  0.0180  0.0244  0.0316  0.0164
3    0.1264  ...  0.0121  0.0036  0.0150  0.0085  0.0073  0.0050  0.0044
4    0.4459  ...  0.0031  0.0054  0.0105  0.0110  0.0015  0.0072  0.0048
..      ...  ...     ...     ...     ...     ...     ...     ...     ...
203  0.2684  ...  0.0116  0.0098  0.0199  0.0033  0.0101  0.0065  0.0115
204  0.2154  ...  0.0061  0.0093  0.0135  0.0063  0.0063  0.0034  0.0032
205  0.2529  ...  0.0160  0.0029  0.0051  0.0062  0.0089  0.0140  0.0138
206  0.2354  ...  0.0086  0.0046  0.0126  0.0036  0.0035  0.0034  0.0079
207  0.2354  ...  0.0146  0.0129  0.0047  0.0039  0.0061  0.0040  0.0036

         58      59  60
0    0.0090  0.0032   R
1    0.0052  0.0044   R
2    0.0095  0.0078   R
3    0.0040  0.0117   R
4    0.0107  0.0094   R
..      ...     ...  ..
203  0.0193  0.0157   M
204  0.0062  0.0067   M
205  0.0077  0.0031   M
206  0.0036  0.0048   M
207  0.0061  0.0115   M

[208 rows x 61 columns]>
```

In [104…

```
df.shape
```

Out[104…

```
(208, 61)
```

In [105…

```
df.columns
```

Out[105…

```
Index([ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11, 12, 13, 14, 15, 16, 17,
       18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35,
       36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 46, 47, 48, 49, 50, 51, 52, 53,
       54, 55, 56, 57, 58, 59, 60],
      dtype='int64')
```

In [106…

```
df[60].value_counts()
```

Out[106…
```
60
M    111
R     97
Name: count, dtype: int64
```

X and y

In [107…
```
X = df.drop(columns=60, axis=1)
y = df[60]
```

Feture Scalling

In [108…
```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

In [109…
```
print(X_train.shape)
print(X_test.shape)
print(y_train.shape)
print(y_test.shape)
```

```
(166, 60)
(42, 60)
(166,)
(42,)
```

In [110…
```
df.head()
```

Out[110…

	0	1	2	3	4	5	6	7	8	9	...	51
0	0.0200	0.0371	0.0428	0.0207	0.0954	0.0986	0.1539	0.1601	0.3109	0.2111	...	0.0027
1	0.0453	0.0523	0.0843	0.0689	0.1183	0.2583	0.2156	0.3481	0.3337	0.2872	...	0.0084
2	0.0262	0.0582	0.1099	0.1083	0.0974	0.2280	0.2431	0.3771	0.5598	0.6194	...	0.0232
3	0.0100	0.0171	0.0623	0.0205	0.0205	0.0368	0.1098	0.1276	0.0598	0.1264	...	0.0121
4	0.0762	0.0666	0.0481	0.0394	0.0590	0.0649	0.1209	0.2467	0.3564	0.4459	...	0.0031

5 rows × 61 columns

In [111…
```
df.shape
```

Out[111…
```
(208, 61)
```

Feature Engineering

In [112…
```
X = df.drop(columns=60)
y = df[60]
```

In [113…
```
X.head()
```

Out[113…

	0	1	2	3	4	5	6	7	8	9	...	50
0	0.0200	0.0371	0.0428	0.0207	0.0954	0.0986	0.1539	0.1601	0.3109	0.2111	...	0.0232
1	0.0453	0.0523	0.0843	0.0689	0.1183	0.2583	0.2156	0.3481	0.3337	0.2872	...	0.0125
2	0.0262	0.0582	0.1099	0.1083	0.0974	0.2280	0.2431	0.3771	0.5598	0.6194	...	0.0033
3	0.0100	0.0171	0.0623	0.0205	0.0205	0.0368	0.1098	0.1276	0.0598	0.1264	...	0.0241
4	0.0762	0.0666	0.0481	0.0394	0.0590	0.0649	0.1209	0.2467	0.3564	0.4459	...	0.0156

5 rows × 60 columns

In [114…
```
y.head()
```

Out[114…
```
0    R
1    R
2    R
3    R
4    R
Name: 60, dtype: object
```

In [115…
```
print(X.shape)
print(y.shape)
```

```
(208, 60)
(208,)
```

Model Building

Train-Test Split

The dataset is divided into training and testing data.

- Training Data : 80%

- Testing Data : 20%

In [116…
```
from sklearn.model_selection import train_test_split
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

In [117…
```
print(X_train.shape)
print(X_test.shape)
```

```
(166, 60)
(42, 60)
```

In [118…
```
from sklearn.linear_model import LogisticRegression
```

Model craeting

In [119…
```
model = LogisticRegression()
```

Model Training

In [120…
```
model.fit(X_train, y_train)
```

Out[120…

LogisticRegression

▶ Parameters

▶ Fitted attributes

A Logistic Regression model is used for classification, The model is trained using the training dataset

Model Evaluation

Prediction

In [121…
```
train_predictions = model.predict(X_train)
test_predictions = model.predict(X_test)
```

In [122…
```
train_predictions
```

Out[122…
```
array(['R', 'M', 'R', 'M', 'M', 'M', 'R', 'R', 'R', 'M', 'R', 'M', 'M',
       'M', 'M', 'M', 'R', 'M', 'M', 'M', 'M', 'M', 'R', 'M', 'R', 'R',
       'R', 'R', 'R', 'R', 'R', 'M', 'R', 'M', 'M', 'R', 'R', 'M', 'R',
       'M', 'R', 'M', 'M', 'M', 'M', 'R', 'R', 'M', 'M', 'R', 'R', 'M',
       'R', 'M', 'R', 'R', 'M', 'M', 'M', 'R', 'M', 'M', 'R', 'R', 'M',
       'M', 'M', 'R', 'R', 'R', 'M', 'M', 'R', 'M', 'R', 'R', 'M', 'M',
       'M', 'R', 'M', 'R', 'R', 'M', 'R', 'M', 'M', 'R', 'M', 'R', 'M',
       'M', 'M', 'R', 'M', 'M', 'R', 'M', 'R', 'M', 'R', 'M', 'R', 'M',
       'M', 'M', 'M', 'M', 'M', 'R', 'M', 'M', 'R', 'M', 'M', 'M', 'R',
       'M', 'R', 'R', 'M', 'M', 'R', 'R', 'M', 'M', 'M', 'M', 'R', 'R',
       'R', 'R', 'M', 'M', 'M', 'R', 'R', 'R', 'R', 'R', 'M', 'M', 'M',
       'R', 'M', 'M', 'R', 'R', 'M', 'M', 'R', 'R', 'R', 'M', 'R', 'R',
       'M', 'M', 'M', 'M', 'R', 'R', 'R', 'R', 'M', 'R'], dtype=object)
```

Evalutions

In [123…
```
from sklearn.metrics import accuracy_score

print("Train Accuracy :", accuracy_score(y_train, train_predictions))
print("Test Accuracy :", accuracy_score(y_test, test_predictions))
```

```
Train Accuracy : 0.8373493975903614
Test Accuracy : 0.7857142857142857
```

In [124…
```
from sklearn.metrics import confusion_matrix

confusion_matrix(y_test, test_predictions)
```

Out[124…
```
array([[19,  7],
       [ 2, 14]], dtype=int64)
```

In [125…
```
from sklearn.metrics import classification_report

print(classification_report(y_test, test_predictions))
```

```
              precision    recall  f1-score   support

           M       0.90      0.73      0.81        26
           R       0.67      0.88      0.76        16

    accuracy                           0.79        42
   macro avg       0.79      0.80      0.78        42
weighted avg       0.81      0.79      0.79        42
```

Conclusion

In this project, a Logistic Regression model was developed to classify sonar signals as Rock or Mine.

The dataset was cleaned, analyzed, and divided into training and testing sets. The model was trained successfully and achieved an accuracy of approximately 79% on the test data.

This project demonstrates how machine learning can be used for binary classification problems.

In [ ]: